

CS3212 Algorithms

Aya Zirikly
09/29/2026

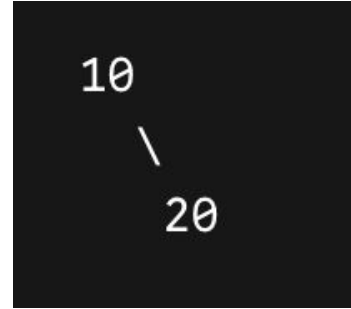
AVL trees

Balance Factor(x) = height(left_child) - height(right_child)

height(x) = 1 + max(height(x.left), height(x.right))

Step 1: Insert 10, 20, 30 (Triggers Right-Right Case)

Insert 10, then 20



Insert 30

```
10 (Balance = -2) [Unbalanced!]
```

```
\
```

```
20
```

```
\
```

```
30
```

Insert 30

Node 10 balance factor becomes **-2** (Left Height 0 – Right Height 2).

Let us look at height and BF for each node

```
10 (Balance = -2) [Unbalanced!]
```

```
\
```

```
20
```

```
\
```

```
30
```

Insert 30

Node 10 balance factor becomes **-2** (Left Height 0 – Right Height 2).

Scenario A: height of empty tree[] is -1 and leaf node is 0

$h(30) = 0$ (leaf node)

$BF(30) = h(L) - h(R) = -1 - (-1) = 0$

$h(20) = 1 + \max(h(L), h(R)) = 1 + \max([], h(30)) = 1 + \max(-1, 0) = 1$

$BF(20) = h(L) - h(R) = -1 - (0) = -1$

$H(10) = 1 + \max([], h(20)) = 1 + \max(-1, 1) = 2$

$BF(10) = h(L) - h(R) = h([]) - h(20) = -1 - 1 = -2$

10 (Balance = -2) [Unbalanced!]



Insert 30

Node 10 balance factor becomes **-2** (Left Height 0 – Right Height 2).

Scenario B: height of empty tree is 0 and leaf node is 1

$h(30) = 1$ (leaf node)

$BF(30) = h(L) - h(R) = 0 - 0 = 0$

$h(20) = 1 + \max(h(L), h(R)) = 1 + \max([], h(30)) = 1 + \max(0, 1) = 2$

$BF(20) = h(L) - h(R) = 0 - 1 = -1$

$H(10) = 1 + \max([], h(20)) = 1 + \max(0, 2) = 3$

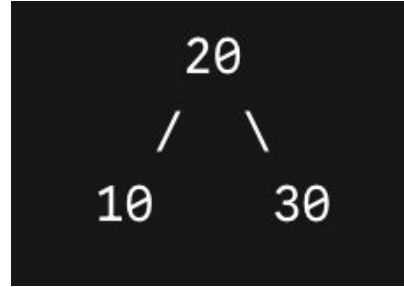
$BF(10) = h(L) - h(R) = h([]) - h(20) = 0 - 2 = -2$

10 (Balance = -2) [Unbalanced!]



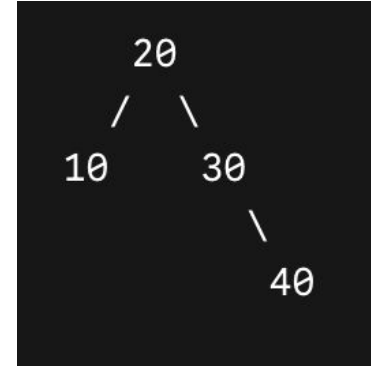
Fix: Left Rotation at 10

Rotate 20 up to become the new root.

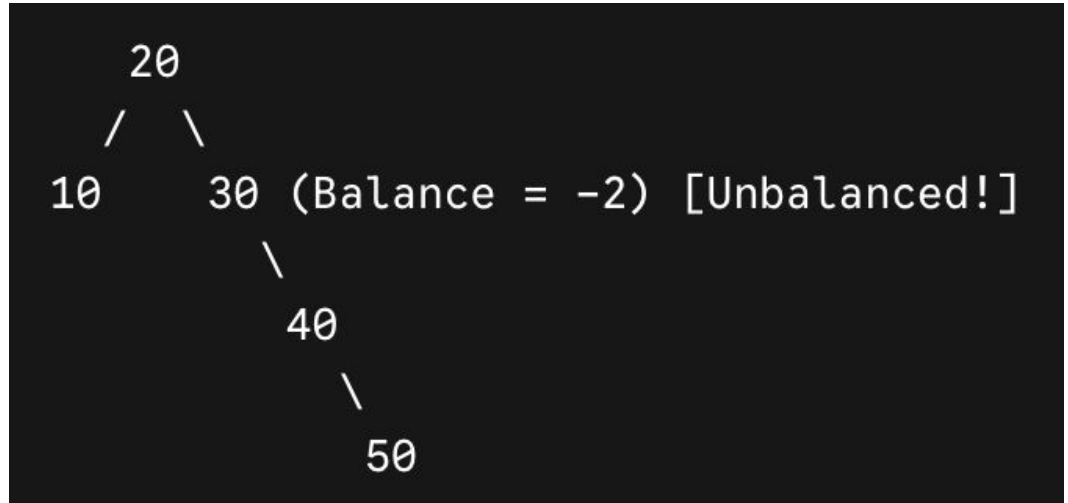


Step 2: Insert 40, 50 (Triggers Right-Right Case)

Insert 40



Insert 50



Node 30 balance factor becomes **-2** (Left Height 0 – Right Height 2).

Calculate height and BF for each node.

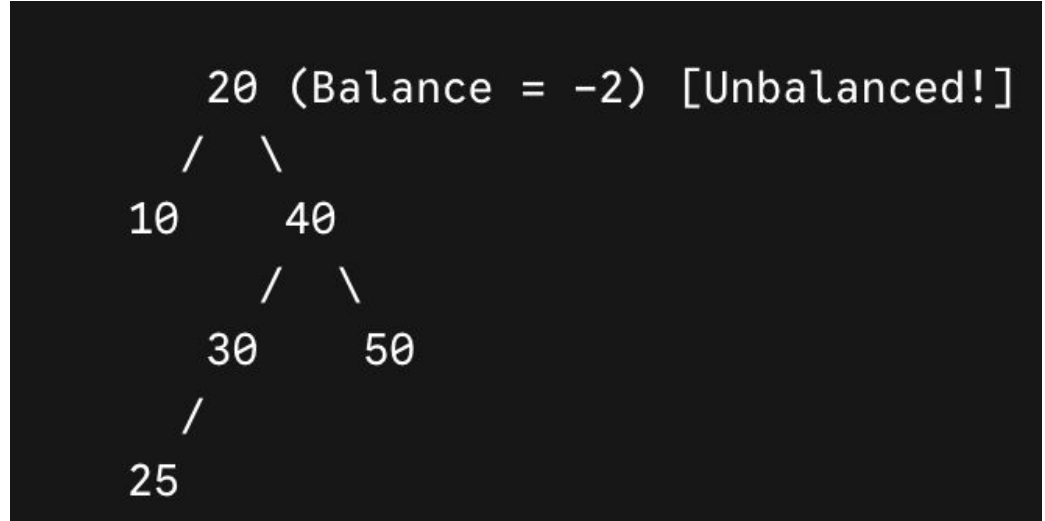
Insert 25 (Triggers Right-Left Case)

Insert 25

Walk down: $25 > 20 \rightarrow 25 < 40 \rightarrow 25 < 30$. Insert 25 as left child of 30.

Node 20 balance factor becomes -2 (Left Height 1 – Right Height 3).

Calculate height and BF for each node.

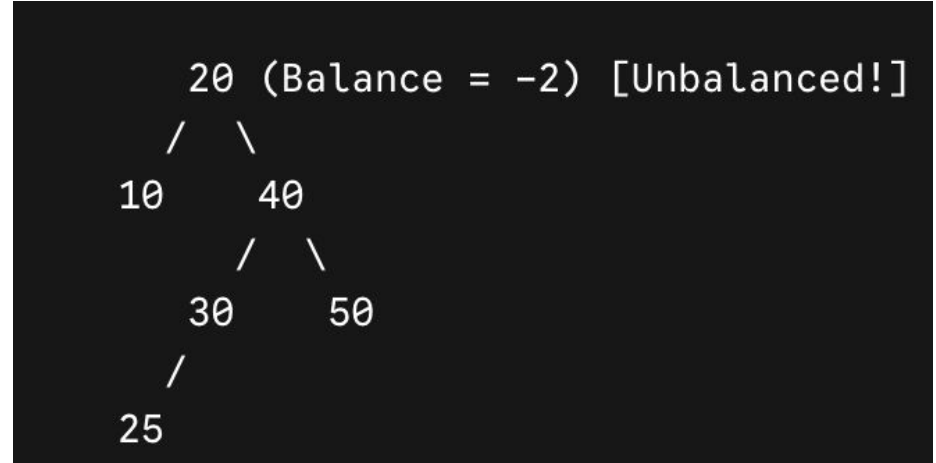


Insert 25 (Triggers Right-Left Case)

Insert 25

Walk down: $25 > 20 \rightarrow 25 < 40 \rightarrow 25 < 30$. Insert 25 as left child of 30.

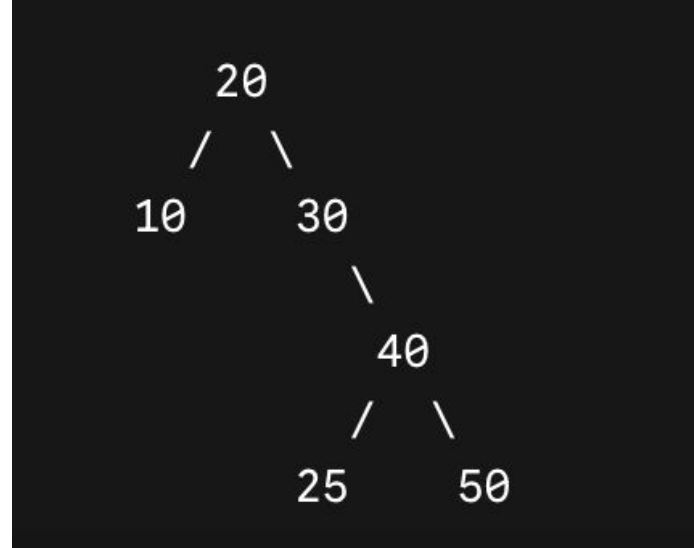
Node 20 balance factor becomes -2 (Left Height 1 – Right Height 3).



Because 25 was inserted into the **left** subtree of 40 and 40 is the **right** child of 20, this is a **Right-Left (RL) Case**.

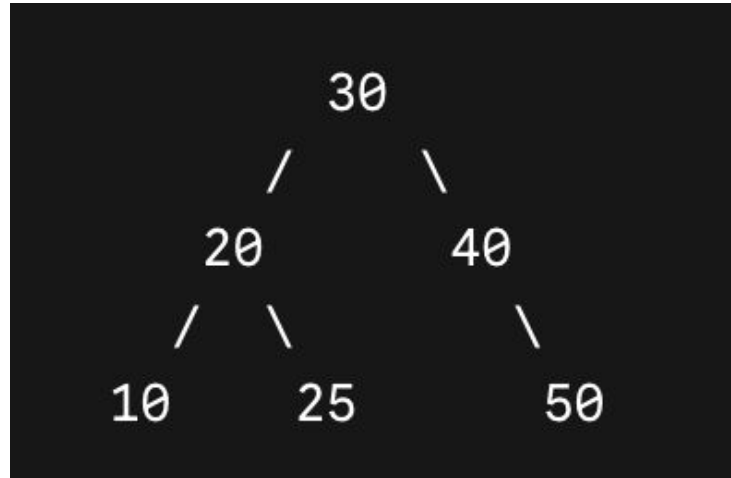
Fix Part 1: Right Rotation at 40 (its right child)

Rotate 30 up



Fix Part 2: Left Rotation at 20

Rotate 30 up to become the new root of the entire tree.



The Physical Reality: Disk I/O vs. Memory Access

RAM Access (In-Memory Data Structures):

- Pointers point to physical memory addresses. Accessing a memory address takes nanoseconds (~10 - 100 ns).
- In RAM, traversing a standard Binary Search Tree (BST) of height 30 requires 30 pointer hops, which executes in a few microseconds.

Disk Access (Secondary Storage Data Structures)

- ❑ Data on a disk cannot be addressed at the individual byte level; it is read and written in blocks/pages (typically 4 KB to 16 KB).
- ❑ Pointers in a disk-based data structure are file offsets or block numbers.
- ❑ Following a pointer to a child node means issuing an I/O command to fetch an entire disk page into RAM.
- ❑ A single disk read takes milliseconds (~5 - 10 ms for mechanical hard drives due to seek time and rotational latency, or ~0.1 ms for SSDs).

Why a Tree of Height 30 Fails on Disk

- ❑ In a standard Binary Search Tree (BST) containing 1 billion keys ($10^9 \sim 2^{30}$), **the tree has a height of ?**

Why a Tree of Height 30 Fails on Disk

- ❑ In a standard Binary Search Tree (BST) containing 1 billion keys ($10^9 \sim 2^{30}$), the tree has a height of $h \sim 30$.
- ❑ If the dataset is too large to fit in RAM and resides on disk:
 - ❑ Each node is stored on disk.
 - ❑ Traversing from a parent node to its left/right child requires following a disk offset pointer.
 - ❑ Because nodes are allocated non-contiguously on disk, each child node access forces a separate block read (1 I/O operation).
 - ❑ 30 levels = 30 sequential disk reads per lookup.

Time per search $\sim 30 \times 10 \text{ ms} = 300 \text{ ms} = 0.3 \text{ seconds}$

How B-Trees Solve This Bottleneck

- ❑ B-Trees are specifically designed to minimize tree height by taking advantage of block-based disk architectures:
- ❑ High Branching Factor (B / Degree t): Instead of storing 1 key and 2 children per node like a binary tree, a B-Tree node is sized to fit an entire disk page (e.g., 4 KB). A single node can hold hundreds or thousands of keys and child pointers (e.g., branching factor $m = 1000$).
- ❑ Drastic Height Reduction: With a branching factor of $m = 1000$, a tree storing 1 billion keys (10^9) has a height of only:

$$h \leq \log_{1000}(10^9) = 3$$

- ❑ Fewer Disk Reads: Instead of 30 distinct disk reads, a lookup requires only 3 disk reads (and the root node is almost always cached in RAM, reducing it to 2 physical I/Os).

Multiway Search Trees (The B-Tree Concept)

Definition: A tree where a single node can store **multiple keys** and have **more than two children** (an m-way search tree).

Key Properties of a B-Tree:

- **Order m:** Every node can have at most m children and m - 1 keys.
- **Self-Balancing:** All leaf nodes are guaranteed to be at the exact same depth.
- **High Branching Factor:** Instead of 2 children, a node might have 100 to 1,000 children.

Impact on Performance:

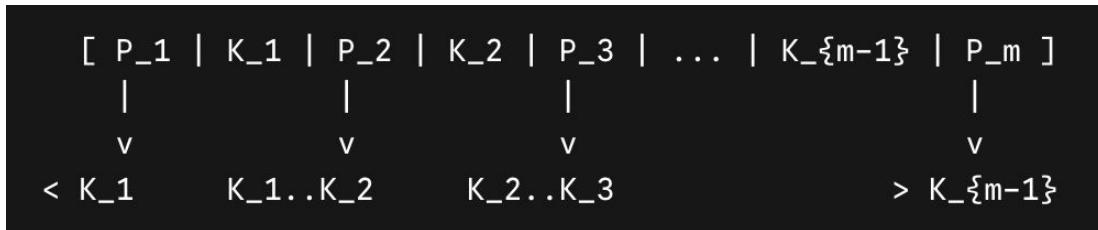
- Tree height drops dramatically: $\text{height} = \log_m(n)$.
- A database with 1 billion items needs a height of only **3 or 4** in a B-Tree, meaning only 3 to 4 disk reads per query!

Structural Alignment

In a B-Tree node, each key K_i separates two child pointers P_i and $P_{(i+1)}$:

- ❑ Pointer P_1 points to keys $< K_1$
- ❑ Pointer P_2 points to keys between K_1 and K_2
- ❑ Pointer P_3 points to keys between K_2 and K_3
- ❑ ...
- ❑ Pointer P_m points to keys $> K_{(m-1)}$

If a node has m child pointers, it creates m distinct ranges to route searches. To define m ranges, you strictly need $m - 1$ boundary keys.



Generalization of Binary Search Trees (BSTs)

Binary Search Tree (Order 2):

- ❑ Maximum children: $m = 2$
- ❑ Maximum keys per node: $m - 1 = 1$ key
- ❑ 1 key creates 2 paths: Left ($< K$) and Right ($> K$).

B-Tree of Order 4:

- ❑ Maximum children: $m = 4$
- ❑ Maximum keys per node: $m - 1 = 3$ keys
- ❑ 3 keys create 4 paths: $< K_1$, between K_1 and K_2 , between K_2 and K_3 , and $> K_3$."

Practical Disk Page Layout

- ❑ In disk implementation, a B-Tree node maps directly to a fixed-size disk block (e.g., 4 KB). The order m is determined by how many key-pointer pairs fit inside that page size:

$$\text{Node Size} = (m * \text{Pointer Size}) + ((m - 1) * \text{Key Size}) \leq \text{Page Size}$$

- ❑ Because m pointers require $m - 1$ keys, every node naturally balances keys and branch addresses without wasting a single byte of disk page space."

Example

Disk Page Size: 4,096 bytes (4 KB)

Key Size (e.g., 64-bit integer): 8 bytes

Pointer Size: 8 bytes

What is the maximum order m we can pick without exceeding one page per node?

Page Size $\geq (m * \text{Pointer Size}) + ((m - 1) * \text{Key Size})$

$4096 \geq (m * 8) + ((m - 1) * 8)$

$4096 \geq 16m - 8$

$16m \leq 4104 \Rightarrow m = 256$

How Search Works in B-Tree

In a standard B-Tree, every key anywhere in the tree acts as a complete entry containing:

- ❑ The key itself (like an ID)
- ❑ The reference to the actual row/record (the data pointer)

Because every key carries its own data pointer, your search can end at any level of the tree:

- ❖ Internal Node Hit: You start searching from the root. If the key you are looking for happens to be stored inside an internal node (even the root node), you don't need to go any lower. You immediately follow that key's associated data pointer to retrieve the full record on disk.
- ❖ Leaf Node Hit: If the key isn't in an internal node, you navigate down through the child pointers until you reach a leaf node, where you locate the key and its data pointer.

B-Tree vs BST

- ❑ **In a Binary Search Tree (BST):** Each node holds only **1 key** and **2 pointers** (~24 bytes). The OS still has to fetch a full 4 KB disk page to read that tiny node. A BST storing 1,000,000 items has a height of around ~20. Traversing it requires up to **20 separate disk reads**, fetching 4 KB every single time just to look at 24 bytes of data.
- ❑ **In a B-Tree:** Each node is engineered to fill an entire 4 KB page. A single node holds **hundreds of keys and pointers** (e.g., $m = 256$). A B-Tree storing 1,000,000 items has a height of only **3**. Traversing it requires at most **3 disk reads**.

B-Tree Pros vs. Cons

Pros:

Fast Best-Case Search: If you are searching for a key that happens to sit high up in the tree (near the root), you can fetch the record in just 1 or 2 disk reads without traversing all the way to the bottom.

Cons:

- ❑ **Smaller Branching Factor:** Because every key in an internal node must share precious disk page space with a data pointer, fewer keys fit inside a single 4 KB block. This increases overall tree height compared to structures like B+ Trees.
- ❑ **Complex In-Order Traversal:** Ranging through sorted data requires moving up and down between parent and child nodes, forcing expensive, non-sequential disk I/Os.

In-Order Traversal

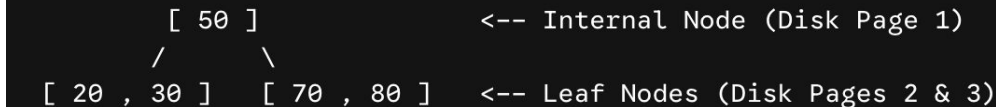
In a standard B-Tree, keys and data records are scattered across **all levels** of the tree (internal nodes and leaf nodes). Because data isn't consolidated in a flat line at the bottom, performing a range query (e.g., *"find all records with IDs between 100 and 500"*) requires an **in-order traversal**, which creates severe disk I/O bottlenecks.

How In-Order Traversal Works in a B-Tree

To retrieve data in sorted order, the algorithm must visit nodes in a strict sequence:

Left Child → Parent Key → Right Child.

To read values from 20 to 80 in order:



1. **Read Disk Page 2** (Leaf) to get 20 and 30.
2. **Jump back to Disk Page 1** (Parent/Internal) to get 50.
3. **Jump forward to Disk Page 3** (Leaf) to get 70 and 80.

B+ Tree (The Modern Database Standard)

- ❑ **What is a B+ Tree?**

- ❑ **Internal Nodes:** Store *only* routing keys and child pointers (no actual data).
- ❑ **Leaf Nodes:** Store *all* actual data or pointers and are **linked together in a doubly linked list**.

- ❑ **Why Databases (e.g., MySQL, PostgreSQL) Prefer B+ Trees:**

- ❑ Higher fan-out (more keys fit into an internal node because there is no data payload).
- ❑ Sequential range scans (e.g., `WHERE age BETWEEN 20 AND 30`) only need to find the first leaf and follow the linked list, avoiding tree re-traversals.

Summary

Feature	Standard B-Tree	B+ Tree
Branching Factor (m)	Lower (data payloads take up space in internal nodes).	Much Higher (internal nodes contain only small keys and pointers).
Point Queries ($O(\log n)$)	Search can terminate early if key is in an internal node.	Must always reach a leaf node (consistent search time).
Range Queries (WHERE age BETWEEN 20 AND 30)	Requires expensive tree traversals (in-order traversal across disk pages).	Trivial. Find the start key at a leaf, then traverse the linked list horizontally across leaves.

Core Operations Overview (B-Tree / B+ Tree)

Search: Traverse down child pointers based on range comparisons inside each node.

Insertion & Node Splitting:

- Insert key into appropriate leaf.
- If node overflows (more than $m - 1$ keys), **split node into two** and push the middle key up to the parent.
- Splits propagate upwards (the tree grows *up*, not down).

Deletion & Node Merging:

- Remove key from leaf.
- If node underflows (fewer than $\text{ceiling}(m / 2) - 1$ keys), borrow keys from siblings or **merge** with a sibling.

B+ Tree —Insert

Leaf-First Insertion: New keys are always inserted directly into a leaf node at the bottom of the tree.

Bottom-Up Growth: B+ Trees grow upward from the leaves toward the root, which guarantees that all leaf nodes remain at the exact same depth.

Copy Up vs. Push Up Rule:

- ❑ Leaf Node Split -> COPY UP: The middle key is copied to the parent because leaf nodes must retain all actual data keys.
- ❑ Internal Node Split -> PUSH UP: The middle key is moved (promoted) to the parent because internal nodes only serve as routing index markers.